GEMMAI : AGENT DÉVELOPPEUR AUTONOME

Rapport d'Ingénierie, Travaux Réalisés sur le Modèle & Objectifs Visés

Cadre du Projet :	Compétition Kaggle *« The Gemma 4 Developer Agent »*	**Cagnotte Globale :**	**100 000 $ USD**
Lead Technique :	Max_Adis (Maker, Développeur & Ingénieur)	**Échéance Finale :**	**25 Novembre 2026**
Modèle Exploité :	Gemma (2B / 7B) quantifié 4-bit (Offline / CPU)	**Contrainte Matérielle :**	Grand public (Max 16 Go RAM, 0 API Cloud)

1. Synthèse Exécutive & Vision Stratégique

Le projet **GemmAI** a été conçu pour répondre au défi de la compétition officielle Kaggle dotée de **100 000 $** : développer un **agent de génie logiciel autonome fonctionnant hors ligne** sur du matériel grand public (16 Go RAM maximum, mode CPU). Propulsé par la famille de modèles ouverts **Gemma de Google**, GemmAI agit comme un ingénieur autonome capable d'explorer un dépôt Git, d'écrire et modifier du code source, d'exécuter des tests en console, et d'auto-corriger ses bugs en boucle fermée sans intervention humaine.

2. Bilan des Travaux Réalisés sur le Modèle & le Moteur

A. Élimination des Limites du Paradigme ReAct Classique

Sur les modèles compacts (SLM) comme Gemma 2B, le paradigme ReAct verbeux traditionnel provoquait des dysfonctionnements critiques : 1) **Syndrome du perroquet** : recopie bête des exemples du prompt, 2) **Boucles infinies** d'exploration sur répertoires vides, et 3) **Plantages de parsing** dus aux conflits de guillemets imbriqués dans les chaînes de caractères.

B. Transition vers l'Architecture Directe « Style Gemini CLI »

Nous avons opéré une **refonte architecturale complète** en adoptant une approche par action directe inspirée de *Gemini CLI* et *Claude Code* :

Critère	Architecture Précédente (ReAct lourd)	Architecture GemmAI Actuelle (Gemini CLI)
Format de sortie	JSON strict complexe ou eval() instable	**Blocs markdown étiquetés directs** (```fichier:...) et streaming propre
Contexte Projet	Outil manuel à relancer à chaque itération	**Injection dynamique automatique** du dossier et des fichiers existants
Fiabilité parsing	Vulnérable aux erreurs de syntaxe Python	**Tolérant à 100%** grâce au *Smart Match* contextuel de secours
Temps de réponse	10 itérations lentes et répétitives	**Action immédiate** en une fraction de seconde avec retour visuel clair
Gestion intention	Exécution confuse même sur un simple 'salut'	**Filtrage intelligent :** Discussion polie vs Mode Développeur actif

3. L'Arsenal Technique & Fonctionnalités Déployées

■ Auto-Correction Autonome (Self-Healing Loop)
En cas d'erreur de commande Bash ou de code Python (Traceback, SyntaxError), GemmAI capture l'erreur et réinterroge le modèle en circuit fermé pour réécrire et re-tester le fichier corrigé sans intervention humaine.

■ Restauration Instantanée (/undo)
Un point de contrôle Git automatique est créé avant chaque opération. L'utilisateur peut taper /undo à tout instant pour réinitialiser le projet à son état antérieur exact (garantie zéro perte de travail).

■ Visualisation Diff Couleur (Style Git)
Lors de toute réécriture d'un fichier existant, GemmAI affiche un diff unifié haute lisibilité : lignes ajoutées en vert (+), lignes supprimées en rouge (-).

■ Analyse & Injection Contextuelle de Code
Quand l'utilisateur mentionne un fichier (ex: *analyse script.py*), l'agent lit automatiquement le fichier sur le disque et injecte son contenu dans le prompt pour une compréhension immédiate du problème.

■ Triple Vérification d'Écriture
Chaque fichier écrit est audité : 1) Présence réelle sur disque, 2) Taille non nulle, 3) Relecture et comparaison exacte du contenu, 4) Compilation syntaxique via python3 -m py_compile.

■ Commande /model Dynamique
Possibilité d'interroger Ollama et de basculer à chaud entre gemma:2b, gemma2:2b ou d'autres modèles sans jamais redémarrer ni modifier le script.

4. Stratégie de Déploiement & Portabilité Solo (Zéro Dépendance)

• **L'installateur One-Line (Script install.sh) :** Permet à n'importe quel développeur sous Linux/macOS d'installer l'agent en une commande unique (curl -fsSL https://.../install.sh | bash). Le script configure l'environnement, vérifie le moteur IA, télécharge le modèle Gemma et crée l'exécutable global /usr/local/bin/gemmai.

• **L'Application Standalone Solo (Format GGUF + llama-cpp) :** Afin de s'affranchir totalement d'Ollama pour l'environnement Kaggle, l'agent peut intégrer le moteur C++ léger llama-cpp-python avec les poids quantifiés en 4-bit dans un fichier unique (.gguf). Un binaire compilé via **PyInstaller** permet d'exécuter GemmAI en solo absolu sans nécessiter l'installation de Python ni de serveur tiers.

5. Objectifs Visés & Feuille de Route Stratégique (Échéance Nov 2026)

Phase	Objectifs & Livrables Techniques	Statut
Phase 1 : Cœur Agent	Architecture interactive directe type Gemini CLI, triple vérification de fichier, streaming ANSI, commande globale.	✔ TERMINÉ
Phase 2 : Robustesse & UX	Boucle d'auto-correction (Self-Healing), Rollback Git instantané (/undo), diffs visuels unifiés, filtrage conversationnel.	✔ TERMINÉ
Phase 3 : Packaging Solo	Script d'installation automatisé install.sh, intégration du runtime local autonome pour exécution hors ligne.	EN COURS
Phase 4 : Benchmarking SWE	Évaluation sur des jeux de test Kaggle (résolution de bugs logiciels réels sur dépôts Git) et optimisation des métriques de succès.	PLANIFIÉ
Phase 5 : Fine-Tuning QLoRA	Adaptation fine des poids de Gemma 2B/7B sur des traces de raisonnement développeur pour maximiser l'autonomie sur cas extrêmes.	PLANIFIÉ
Phase 6 : Soumission Finale	Dépôt officiel du Notebook Kaggle et de la documentation complète pour le grand prix de **100 000 $ USD**.	**OBJECTIF FINAL (25 Nov 2026)**

Document rédigé par : Max_Adis • Lead Technique & Ingénieur Développeur
Projet GemmAI — Tous droits réservés • Compétition Kaggle 2026